

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Akmallullail Sya'ban NIM. 2310817310010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Akmallullail Sya'ban
NIM : 2310817310010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	6
B. Output Program	29
C. Pembahasan	35
XML	35
Jetpack Compose	39
D. Tautan Git.....	42
SOAL 2.....	43
A. Pembahasan	43

DAFTAR GAMBAR

Gambar 1. Tampilan Awal UI XML	29
Gambar 2. Tampilan Ketika di Scroll XML.....	29
Gambar 3. Tampilan Tombol Detail Ditekan XML	30
Gambar 4. Tampilan Awal UI di Rotate XML.....	30
Gambar 5. Tampilan Detail UI di Rotate XML	30
Gambar 6. Tombol IMDB di Tekan	31
Gambar 7. Tampilan Awal UI Jetpack Compose	31
Gambar 8. Tampilan UI Ketika di Scroll Jetpack Compose	32
Gambar 9. Tampilan Detail UI Jetpack Compose.....	32
Gambar 10. Tampilan Awal UI di Rotate Jetpack Compose	33
Gambar 11. Tampilan Detail UI di Rotate Jetpack Compose	33
Gambar 12. Tampilan Ketika Tombol IMDB di Tekan Jetpack Compose	33
Gambar 13. Gambar Debugger Di XML.....	34
Gambar 14. Gambar Debugger Di Jetpack Compose.....	34

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt App XML	6
Tabel 2. Source Code Movie.kt App XML	7
Tabel 3. Source Code ListMovieAdapter.kt App XML	7
Tabel 4. Source Code HomeFragment.kt App XML	8
Tabel 5. Source Code DetailFragmen.kt App XML	11
Tabel 6. Source Code MovieViewModel.kt App XML	12
Tabel 7. Source Code MovieViewModelFactory.kt App XML	13
Tabel 8. Source Code MyApplication.kt App XML	13
Tabel 9. Source Layout XML activity_main.xml	14
Tabel 10. Source Layout XML fragment_detail.xml	14
Tabel 11. Source Layout XML fragment_home.xml	16
Tabel 12. Source Layout XML item_movie.xml	16
Tabel 13. Source Strings.xmlJetpack Compose	19
Tabel 14. Source Code DataMovie.kt Jetpack Compose	20
Tabel 15. Source ListMovie.kt Jetpack Compose	21
Tabel 16. Source DetailActivity.kt Jetpack Compose	21
Tabel 17. Source MovieViewModel.kt Jetpack Compose	23
Tabel 18. Source MovieViewModelFactory.kt Jetpack Compose	24
Tabel 19. Source MainActivity.kt Jetpack Compose	25

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

XML:

MainActivity.kt

Tabel 1. Source Code MainActivity.kt App XML

1	package com.example.movielistxml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import timber.log.Timber
6	
7	class MainActivity : AppCompatActivity() {
8	override fun onCreate(savedInstanceState: Bundle?) {
9	super.onCreate(savedInstanceState)
10	setContentView(R.layout.activity_main)
11	
12	val fragmentManager = supportFragmentManager
13	val homeFragment = HomeFragment()
14	val fragment =
	fragmentManager.findFragmentByTag(HomeFragment::class.java.simpleName)
15	if (fragment !is HomeFragment) {
16	Timber.d("MyFragment", "Fragment Name : " +
	HomeFragment::class.java.simpleName)
17	fragmentManager
18	.beginTransaction()

19	<code>.add(R.id.frame_container, homeFragment,</code>
20	<code>HomeFragment::class.java.simpleName)</code>
21	<code>.commit()</code>
22	<code>}</code>
23	<code>}</code>

Movie.kt

Tabel 2. Source Code Movie.kt App XML

1	<code>package com.example.movielistxml</code>
2	
3	<code>import android.os.Parcelable</code>
4	<code>import kotlinx.parcelize.Parcelize</code>
5	
6	<code>@Parcelize</code>
7	<code>data class Movie (</code>
8	<code> val title: String,</code>
9	<code> val url: String,</code>
10	<code> val image: Int,</code>
11	<code> val plot: String,</code>
12	<code> val years: String,</code>
13	<code> val desc: String</code>
14	<code>): Parcelable</code>

ListMovieAdapter.kt

Tabel 3. Source Code ListMovieAdapter.kt App XML

1	<code>package com.example.movielistxml</code>
2	
3	<code>import android.view.LayoutInflater</code>
4	<code>import android.view.View</code>
5	<code>import android.view.ViewGroup</code>
6	<code>import android.widget.Button</code>
7	<code>import android.widget.ImageView</code>
8	<code>import android.widget.TextView</code>
9	<code>import androidx.recyclerview.widget.RecyclerView</code>
10	
11	<code>class ListMovieAdapter (</code>
12	<code> private val listMovie: ArrayList<Movie>,</code>
13	<code> private val onImdbClick: (String) -> Unit,</code>
14	<code> private val onDetailClick: (String, Int, String, String,</code>
	<code>String) -> Unit</code>
15	<code>): RecyclerView.Adapter<ListMovieAdapter.ListViewHolder>() {</code>
16	
17	<code> class ListViewHolder(itemView: View) :</code>
	<code>RecyclerView.ViewHolder(itemView) {</code>
18	<code> val imgPoster: ImageView =</code>

```

19     itemView.findViewById(R.id.img_poster)
        val tvName: TextView =
20     itemView.findViewById(R.id.tv_item_title)
        val years: TextView =
21     itemView.findViewById(R.id.tv_item_years)
        val plot: TextView =
22     itemView.findViewById(R.id.tv_item_plot)
        val desc: TextView =
23     itemView.findViewById(R.id.tv_item_desc)
        val btnImdb: Button =
24     itemView.findViewById(R.id.btn_imdb)
        val btnDetail: Button =
25     itemView.findViewById(R.id.btn_detail)
    }
26     override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): ViewHolder{
27         val view: View =
LayoutInflater.from(parent.context).inflate(R.layout.item_mov
ie, parent, false)
28         return ViewHolder(view)
29     }
30     override fun getItemCount(): Int = listMovie.size
31
32     override fun onBindViewHolder(holder: ViewHolder,
position: Int){
33         val (title, url, image, plot, years, desc) =
listMovie[position]
34         holder.tvName.text = title
35         holder.plot.text = plot
36         holder.imgPoster.setImageResource(image)
37         holder.years.text = years
38         holder.desc.text = desc
39         holder.btnImdb.setOnClickListener{onImdbClick(url)}
40         holder.btnDetail.setOnClickListener{onDetailClick(title,
image, plot, years, desc)}
41     }
42
43     fun updateData(newList: List<Movie>) {
44         listMovie.clear()
45         listMovie.addAll(newList)
46         notifyDataSetChanged()
47     }
48 }

```

HomeFragment.kt

Tabel 4. Source Code HomeFragment.kt App XML

1	package com.example.movieslistxml
2	


```

3  import android.content.Intent
4  import android.net.Uri
5  import android.os.Bundle
6  import androidx.fragment.app.Fragment
7  import android.view.LayoutInflater
8  import android.view.View
9  import android.view.ViewGroup
10 import androidx.fragment.app.viewModels
11 import androidx.lifecycle.lifecyclescope
12 import androidx.recyclerview.widget.LinearLayoutManager
13 import
14 com.example.movielistxml.databinding.FragmentHomeBinding
15 import kotlinx.coroutines.flow.collectLatest
16 import kotlinx.coroutines.launch
17 import timber.log.Timber
18
19 class HomeFragment : Fragment() {
20     private var _binding: FragmentHomeBinding? = null
21     private val binding get() = _binding!!
22
23     private lateinit var movieAdapter: ListMovieAdapter
24
25     private val viewModel: MovieViewModel by viewModels {
26         MovieViewModelFactory("Akmal")
27     }
28
29     override fun onCreateView(
30         inflater: LayoutInflater, container: ViewGroup?,
31         savedInstanceState: Bundle?
32     ): View {
33         _binding = FragmentHomeBinding.inflate(inflater,
34 container, false)
35         return binding.root
36     }
37
38     override fun onViewCreated(view: View,
39 savedInstanceState: Bundle?) {
40         super.onViewCreated(view, savedInstanceState)
41
42         setupRecyclerView()
43
44         val movies = getListMovie()
45         viewModel.setMovieList(movies)
46
47         viewLifecycleOwner.lifecycleScope.launch {
48             viewModel.movieList.collectLatest { list ->
49                 Timber.i("Movie List updated with
50 ${list.size} items")
51                 movieAdapter.updateData(ArrayList(list))
52             }
53         }
54     }
55 }

```

```

51         viewLifecycleOwner.lifecycleScope.launch {
52             viewModel.selectedMovie.collectLatest { movie ->
53                 movie?.let {
54                     Timber.i("Movie selected:
55                     ${movie.title}")
56                     val detailFragment =
57                     DetailFragment().apply {
58                         arguments = Bundle().apply {
59                             putString("EXTRA_NAME",
60                                 movie.title)
61                             putInt("EXTRA_PHOTO",
62                                 movie.image)
63                             putString("EXTRA_YEARS",
64                                 movie.years)
65                             putString("EXTRA_PLOT",
66                                 movie.plot)
67                             putString("EXTRA_DESC",
68                                 movie.desc)
69                         }
70                     }
71                     parentFragmentManager.beginTransaction()
72                         .replace(R.id.frame_container,
73                             detailFragment)
74                         .addToBackStack(null)
75                         .commit()
76                 }
77             }
78         }
79     }
80
81     private fun setupRecyclerView(){
82         movieAdapter = ListMovieAdapter(
83             arrayListOf(),
84             onImdbClick = {url ->
85                 Timber.i("IMDB clicked with URL: $url")
86                 val intent = Intent(Intent.ACTION_VIEW,
87                     Uri.parse(url))
88                 startActivity(intent)
89             },
90             onDetailClick = {title, _, _, _, _ ->
91                 Timber.i("Detail clicked with title:
92                 $title")
93                 val movie = viewModel.movieList.value.find {
94                     it.title == title }
95                 movie?.let {
96                     viewModel.selectMovie(it)
97                 }
98             }
99         )
100         binding.rvMovie.apply {
101             layoutManager = LinearLayoutManager(context)
102             adapter = movieAdapter

```

```

92         setHasFixedSize(true)
93     }
94 }
95
96     private fun getListMovie(): ArrayList<Movie>{
97         val dataMovie =
98             resources.getStringArray(R.array.data_title)
99         val dataLink =
100             resources.getStringArray(R.array.data_link)
101         val dataImage =
102             resources.obtainTypedArray(R.array.data_image)
103         val dataPlot =
104             resources.getStringArray(R.array.plot)
105         val dataYear =
106             resources.getStringArray(R.array.data_years)
107         val dataDesc =
108             resources.getStringArray(R.array.data_description)
109         val listMovie = ArrayList<Movie>()
110         for (i in dataMovie.indices){
111             val movie = Movie(dataMovie[i], dataLink[i],
112                 dataImage.getResourceId(i, -1), dataPlot[i], dataYear[i],
113                 dataDesc[i])
114             listMovie.add(movie)
115         }
116         dataImage.recycle()
117         return listMovie
118     }
119
120     override fun onDestroyView() {
121         super.onDestroyView()
122         _binding = null
123     }
124 }

```

DetailFragment.kt

Tabel 5. Source Code DetailFragmen.kt App XML

```

1 package com.example.movielistxml
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import
9     com.example.movielistxml.databinding.FragmentDetailBinding
10
11 class DetailFragment : Fragment() {
12     private var _binding: FragmentDetailBinding? = null

```

```

13     private val binding get() = _binding!!
14
15     override fun onCreateView(
16         inflater: LayoutInflater, container: ViewGroup?,
17         savedInstanceState: Bundle?
18     ): View {
19         _binding = FragmentDetailBinding.inflate(inflater,
20 container, false)
21
22         val title = arguments?.getString("EXTRA_NAME")
23         val image = arguments?.getInt("EXTRA_PHOTO")
24         val years = arguments?.getString("EXTRA_YEARS")
25         val plot = arguments?.getString("EXTRA_PLOT")
26         val desc = arguments?.getString("EXTRA_DESC")
27
28         binding.tvItemTitle.text = title
29         binding.tvItemPlot.text = years
30         binding.tvItemYears.text = plot
31         binding.tvItemDesc.text = desc
32         image?.let {
33             binding.imgPoster.setImageResource(it)
34         }
35         return binding.root
36     }
37
38     override fun onDestroyView() {
39         super.onDestroyView()
40         _binding = null
41     }

```

MovieViewModel.kt

Tabel 6. Source Code MovieViewModel.kt App XML

```

1 package com.example.movielistxml
2
3 import androidx.lifecycle.ViewModel
4 import kotlinx.coroutines.flow.MutableStateFlow
5 import kotlinx.coroutines.flow.StateFlow
6 import timber.log.Timber
7
8 class MovieViewModel(private val username: String):
9     ViewModel() {
10         private val _movieList =
11             MutableStateFlow<List<Movie>>(emptyList())
12         val movieList: StateFlow<List<Movie>> get() = _movieList
13
14         private val _selectedMovie =
15             MutableStateFlow<Movie?>(null)
16         val selectedMovie: StateFlow<Movie?> get() =

```

```

14  _selectedMovie
15      init {
16          Timber.i("MovieViewModel created with username:
$username")
17      }
18
19      fun setMovieList(movies: List<Movie>){
20          Timber.i("Movie List loaded With: ${movies.size}")
21          _movieList.value = movies
22      }
23
24      fun selectMovie(movie: Movie) {
25          Timber.i("Movie selected: ${movie.title}")
26          _selectedMovie.value = movie
27      }
28  }

```

MovieViewModelFactory.kt

Tabel 7. Source Code MovieViewModelFactory.kt App XML

```

1  package com.example.movieslistxml
2
3  import androidx.lifecycle.ViewModel
4
5  class MovieViewModelFactory(private val username: String) :
    androidx.lifecycle.ViewModelProvider.Factory {
6      override fun <T : ViewModel> create(modelClass:
    Class<T>): T {
7
8          if(modelClass.isAssignableFrom(MovieViewModel::class.java)) {
9              @Suppress("UNCHECKED_CAST")
10             return MovieViewModel(username) as T
11         }
12         throw IllegalArgumentException("Unknown ViewModel
    class")
13     }

```

MyApplication.kt

Tabel 8. Source Code MyApplication.kt App XML

```

1  package com.example.movieslistxml
2
3  import android.app.Application
4  import timber.log.Timber
5
6  class MyApplication : Application() {
7      override fun onCreate() {

```

8	super.onCreate()
9	if (BuildConfig.DEBUG) {
10	Timber.plant(Timber.DebugTree())
11	}
12	}
13	}

activity_main.xml

Tabel 9. Source Layout XML activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/android"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/frame_container"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	tools:context=".MainActivity">
10	</FrameLayout>

fragment_detail.xml

Tabel 10. Source Layout XML fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:tools="http://schemas.android.com/tools"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	tools:context=".DetailFragment"
8	android:layout_margin="10dp">
9	
10	<ImageView
11	android:id="@+id/img_poster"
12	android:layout_width="wrap_content"
13	android:layout_height="400dp"
14	android:layout_marginTop="12dp"
15	android:contentDescription="Poster"
16	tools:src="@drawable/nwh"
17	app:layout_constraintTop_toTopOf="parent"
18	app:layout_constraintStart_toStartOf="parent"
19	/>
20	
21	<TextView
22	android:id="@+id/tv_item_title"
23	android:layout_width="350dp"
24	android:layout_height="wrap_content"

```

25         tools:text="Ini Judul"
26         app:layout_constraintTop_toBottomOf="@id/img_poster"
27         app:layout_constraintStart_toStartOf="parent"
28         android:textSize="24sp"
29         android:textStyle="bold"
30         android:textColor="@color/black"
31     />
32
33     <TextView
34         android:id="@+id/tv_item_years"
35         android:layout_width="wrap_content"
36         android:layout_height="wrap_content"
37         android:layout_marginTop="4dp"
38         tools:text="2020"
39         android:textSize="20sp"
40         app:layout_constraintEnd_toEndOf="parent"
41         app:layout_constraintHorizontal_bias="1.0"
42         app:layout_constraintStart_toEndOf="@id/tv_item_title"
43         app:layout_constraintTop_toBottomOf="@id/img_poster"
44     />
45
46     <TextView
47         android:id="@+id/tv_item_plot"
48         android:layout_width="wrap_content"
49         android:layout_height="wrap_content"
50         tools:text="Plot :"
51         app:layout_constraintStart_toStartOf="parent"
52         app:layout_constraintTop_toBottomOf="@id/tv_item_title"
53         android:textSize="16sp"
54         android:textStyle="bold"
55         android:textColor="@color/black"
56     />
57
58     <TextView
59         android:id="@+id/tv_item_desc"
60         android:layout_width="wrap_content"
61         android:layout_height="wrap_content"
62         tools:text="deskripsi"
63         app:layout_constraintTop_toBottomOf="@id/tv_item_plot"
64         app:layout_constraintStart_toStartOf="parent"
65         android:textSize="16sp"
66         android:textStyle="bold"
67         android:textColor="@color/black"
68     />
69 </androidx.constraintlayout.widget.ConstraintLayout>

```

Tabel 11. Source Layout XML fragment_home.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".HomeFragment">
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/rv_movie"
11	android:layout_width="0dp"
12	android:layout_height="0dp"
13	android:layout_margin="15dp"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintTop_toTopOf="parent" />
18	
19	</androidx.constraintlayout.widget.ConstraintLayout>

item_movie.xml

Tabel 12. Source Layout XML item_movie.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:card_view="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/card_view"
7	android:layout_width="match_parent"
8	android:layout_height="wrap_content"
9	android:layout_gravity="center"
10	android:layout_marginStart="8dp"
11	android:layout_marginTop="4dp"
12	android:layout_marginEnd="8dp"
13	android:layout_marginBottom="4dp"
14	card_view:cardCornerRadius="4dp">
15	<androidx.constraintlayout.widget.ConstraintLayout
16	android:layout_width="match_parent"
17	android:layout_height="wrap_content"
18	android:padding="8dp"
19	android:background="@color/gray">
20	
21	<ImageView
22	android:id="@+id/img_poster"
23	android:layout_width="70dp"
24	android:layout_height="100dp"


```

25         android:scaleType="centerCrop"
26     card_view:layout_constraintBottom_toBottomOf="parent"
27     card_view:layout_constraintStart_toStartOf="parent"
28         card_view:layout_constraintTop_toTopOf="parent"
29         tools:src="@drawable/nwh" />
30
31     <TextView
32         android:id="@+id/tv_item_title"
33         android:layout_width="200dp"
34         android:layout_height="wrap_content"
35         android:textColor="@color/white"
36         android:layout_marginStart="12dp"
37         android:textSize="24sp"
38         android:textStyle="bold"
39         tools:layout_editor_absoluteX="78dp"
40         tools:layout_editor_absoluteY="2dp"
41
42     card_view:layout_constraintStart_toEndOf="@id/img_poster"
43     card_view:layout_constraintEnd_toStartOf="@id/tv_item_years"
44         tools:text="Ini Judul" />
45
46     <TextView
47         android:id="@+id/tv_item_years"
48         android:layout_width="wrap_content"
49         android:layout_height="wrap_content"
50         android:layout_marginStart="16dp"
51         android:layout_marginTop="12dp"
52         tools:text="2020"
53         android:textColor="@color/white"
54         card_view:layout_constraintTop_toTopOf="parent"
55
56     card_view:layout_constraintStart_toEndOf="@id/tv_item_title"
57         tools:layout_editor_absoluteY="29dp" />
58
59     <TextView
60         android:id="@+id/tv_item_plot"
61         android:layout_width="wrap_content"
62         android:layout_height="wrap_content"
63         android:layout_marginTop="8dp"
64         android:layout_marginStart="12dp"
65         tools:text="Plot :"
66         android:textColor="@color/white"
67         android:textSize="15sp"
68         android:textStyle="bold"
69
70     card_view:layout_constraintStart_toEndOf="@id/img_poster"
71     card_view:layout_constraintTop_toBottomOf="@id/tv_item_title"
72 />

```

```

69
70         <TextView
71             android:id="@+id/tv_item_desc"
72             android:layout_width="0dp"
73             android:layout_height="wrap_content"
74             android:layout_marginStart="12dp"
75             android:textColor="@color/white"
76
77     card_view:layout_constraintTop_toTopOf="@id/tv_item_plot"
78     card_view:layout_constraintStart_toEndOf="@id/tv_item_plot"
79     card_view:layout_constraintEnd_toEndOf="parent"
80     tools:layout_editor_absoluteY="42dp"
81     tools:text="Deskripsi Film" />
82
83     <Button
84         android:id="@+id/btn_detail"
85         android:layout_width="wrap_content"
86         android:layout_height="wrap_content"
87         android:layout_marginTop="4dp"
88         android:text="@string/btn_detail"
89         card_view:layout_constraintEnd_toEndOf="parent"
90
91     card_view:layout_constraintHorizontal_bias="0.508"
92     card_view:layout_constraintStart_toEndOf="@id/btn_imdb"
93     card_view:layout_constraintTop_toBottomOf="@+id/tv_item_desc"
94     />
95
96     <Button
97         android:id="@+id/btn_imdb"
98         android:layout_width="wrap_content"
99         android:layout_height="wrap_content"
100        android:layout_marginStart="144dp"
101        android:layout_marginTop="4dp"
102        android:text="@string/btn_imdb"
103
104    card_view:layout_constraintStart_toStartOf="parent"
105    card_view:layout_constraintTop_toBottomOf="@id/tv_item_desc"
106    />
107
108    </androidx.constraintlayout.widget.ConstraintLayout>
109    </androidx.cardview.widget.CardView>

```

Compose:

string.xml

Tabel 13. Source Strings.xmlJetpack Compose

```

<resources>
    <string name="app_name">Movie List</string>

    <string name="title1">Iron Man 3</string>
    <string name="title2">Captain America: Civil War</string>
    <string name="title3">Avengers: Endgame</string>
    <string name="title4">Avengers: Infinity War</string>
    <string name="title5">Spider-man: No Way Home</string>

    <string name="releaseDate1">2013</string>
    <string name="releaseDate2">2016</string>
    <string name="releaseDate3">2019</string>
    <string name="releaseDate4">2018</string>
    <string name="releaseDate5">2021</string>

    <string name="desc1">Brilliant industrialist Tony Stark /
    Iron Man faces an enemy that knows no bounds. Tony world is
    destroyed and he investigates who is responsible for the
    destruction. Armed with his tools and instincts, Tony sets out
    to save those he loves. An old question comes back to haunt
    him; did humans make the Iron Man suit or did the suit define
    his identity as a human?</string>
    <string name="desc2">The heavy civilian casualties have led
    the government to control the superheroes. This heated
    situation has Iron Man and Captain America camps opposing each
    other. The battle is inevitable.</string>
    <string name="desc3">The remaining Avengers try to reverse
    Thanos actions that destroyed half the universe. They travel
    through time to collect the Infinity Stones, then use the
    energy of the stones to go back in time and change the events
    that caused the Snap.</string>
    <string name="desc4">The Avengers and their allies must be
    willing to sacrifice everything in an attempt to defeat the
    powerful Thanos before he succeeds in destroying the
    universe.</string>
    <string name="desc5">For the first time in Spider-Man big-
    screen history, the affable hero true identity is revealed,
    causing his superpower responsibilities to clash with his
    normal life, and putting everyone closest to him at
    risk.</string>

    <string name="description1">Brilliant industrialist Tony
    Stark / Iron Man faces an enemy that knows no bounds. Tony
    world is destroyed and he investigates who is responsible for
    the destruction. Armed with his tools and instincts, Tony sets
    out to save those he loves. An old question comes back to haunt
    him; did humans make the Iron Man suit or did the suit define
    his identity as a human?</string>
    <string name="description2">The heavy civilian casualties
    have led the government to control the superheroes. This heated

```

```

situation has Iron Man and Captain America camps opposing each
other. The battle is inevitable.</string>
    <string name="description3">The remaining Avengers try to
reverse Thanos actions that destroyed half the universe. They
travel through time to collect the Infinity Stones, then use
the energy of the stones to go back in time and change the
events that caused the Snap.</string>
    <string name="description4">The Avengers and their allies
must be willing to sacrifice everything in an attempt to defeat
the powerful Thanos before he succeeds in destroying the
universe.</string>
    <string name="description5">For the first time in Spider-
Man big-screen history, the affable hero true identity is
revealed, causing his superpower responsibilities to clash with
his normal life, and putting everyone closest to him at
risk.</string>
</resources>

```

DataMovie.kt

Tabel 14. Source Code DataMovie.kt Jetpack Compose

```

1 package com.example.mlistcompose.data
2
3 import com.example.mlistcompose.R
4 import com.example.mlistcompose.model.ListMovie
5
6 class DataMovie{
7     fun loadMovieList(): List<ListMovie> = listOf(
8         ListMovie(R.string.title1,
9             "https://www.imdb.com/title/tt1300854/?ref_=nv_sr_srsrg_0_tt_8
10             _nm_0_in_0_q_Iron%2520Man%25203" , R.drawable.ironman3,
11             R.string.releaseDate1, R.string.desc1,
12             R.string.description1),
13         ListMovie(R.string.title2,
14             "https://www.imdb.com/title/tt3498820/?ref_=nv_sr_srsrg_0_tt_6
15             _nm_0_in_0_q_captain%2520civil%2520war", R.drawable.civilwar,
16             R.string.releaseDate2, R.string.desc2,
17             R.string.description2),
18         ListMovie(R.string.title3,
19             "https://www.imdb.com/title/tt4154756/?ref_=nv_sr_srsrg_0_tt_8
20             _nm_0_in_0_q_infinity%2520war", R.drawable.infinitywar,
21             R.string.releaseDate3, R.string.desc3,
22             R.string.description3),
23         ListMovie(R.string.title4,
24             "https://www.imdb.com/title/tt4154796/?ref_=nv_sr_srsrg_0_tt_8
25             _nm_0_in_0_q_endgame", R.drawable.endgame,
26             R.string.releaseDate4, R.string.desc4,
27             R.string.description4),
28         ListMovie(R.string.title5,

```

	"https://www.imdb.com/title/tt10872600/?ref_=nv_sr_srsq_0_tt_8_nm_0_in_0_q_no%2520way", R.drawable.nwh, R.string.releaseDate5, R.string.desc5, R.string.description5)
13	)
14	}

ListMovie.kt

Tabel 15. Source ListMovie.kt Jetpack Compose

1	package com.example.mlistcompose.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class ListMovie(
7	@StringRes val title: Int,
8	val url: String,
9	@DrawableRes val image: Int,
10	@StringRes val years: Int,
11	@StringRes val desc: Int,
12	@StringRes val description: Int
13	)

DetailActivity.kt

Tabel 16. Source DetailActivity.kt Jetpack Compose

1	package com.example.mlistcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.Image
7	import androidx.compose.material3.Surface
8	import androidx.compose.material3.Text
9	import androidx.compose.ui.Alignment
10	import androidx.compose.ui.Modifier
11	import androidx.compose.foundation.layout.*
12	import androidx.compose.ui.graphics.Color
13	import androidx.compose.ui.res.painterResource
14	import androidx.compose.ui.res.stringResource
15	import androidx.compose.ui.text.TextStyle
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	import com.example.mlistcompose.ui.theme.MListComposeTheme
20	
21	class DetailActivity : ComponentActivity() {
22	override fun onCreate(savedInstanceState: Bundle?) {
23	super.onCreate(savedInstanceState)


```

68 .align(Alignment.CenterVertically)
69     )
70     }
71     Text(
72         text = "Plot :",
73         style = TextStyle(
74             fontSize = 16.sp,
75             fontWeight =
76             FontWeight.Bold,
77             color = Color.White
78         )
79     )
80     Text(
81         text =
82         stringResource(description),
83         style = TextStyle(
84             fontSize = 16.sp,
85             color = Color.White
86         )
87     )
88     }
89     }
90 }
91 }

```

MovieViewModel.kt

Tabel 17. Source MovieViewModel.kt Jetpack Compose

```

1 package com.example.mlistcompose
2
3 import androidx.lifecycle.ViewModel
4 import com.example.mlistcompose.data.DataMovie
5 import com.example.mlistcompose.model.ListMovie
6 import kotlinx.coroutines.flow.MutableStateFlow
7 import kotlinx.coroutines.flow.StateFlow
8 import kotlinx.coroutines.flow.asStateFlow
9 import timber.log.Timber
10
11 class MovieViewModel(someParam: String) : ViewModel() {
12     private val _movies =
13     MutableStateFlow<List<ListMovie>>(emptyList())
14     val movies: StateFlow<List<ListMovie>> =
15     _movies.asStateFlow()
16
17     private val _navigateToUrl =
18     MutableStateFlow<String?>(null)
19     val navigateToUrl: StateFlow<String?> =
20     _navigateToUrl.asStateFlow()

```

```

17
18     private val _navigateToDetail =
MutableStateFlow<ListMovie?>(null)
19     val navigateToDetail: StateFlow<ListMovie?> =
_navigateToDetail.asStateFlow()
20     init {
21         Timber.d("Loading movie list with param: $someParam")
22         _movies.value = DataMovie().loadMovieList()
23         Timber.d("Movie list loaded with
${_movies.value.size} items")
24     }
25
26     fun onLinkClicked(url: String) {
27         Timber.d("URL clicked: $url")
28         _navigateToUrl.value = url
29     }
30
31     fun onDetailClicked(movie: ListMovie) {
32         Timber.d("Detail clicked for movie: ${movie.title}")
33         _navigateToDetail.value = movie
34     }
35
36     fun onUrlNavigated() {
37         _navigateToUrl.value = null
38     }
39
40     fun onDetailNavigated() {
41         _navigateToDetail.value = null
42     }
43 }

```

MovieViewModelFactory.kt

Tabel 18. Source MovieViewModelFactory.kt Jetpack Compose

```

1 package com.example.mlistcompose
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5
6 class MovieViewModelFactory(private val someParam: String) :
ViewModelProvider.Factory {
7     override fun <T : ViewModel> create(modelClass:
Class<T>): T {
8         if
9         (modelClass.isAssignableFrom(MovieViewModel::class.java)) {
10             @Suppress("UNCHECKED_CAST")
11             return MovieViewModel(someParam) as T
12         }
13         throw IllegalArgumentException("Unknown ViewModel
class")

```


13	}
14	}

MainActivity.kt

Tabel 19. Source MainActivity.kt Jetpack Compose

1	package com.example.mlistcompose
2	
3	import android.content.Intent
4	import android.os.Bundle
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.activity.enableEdgeToEdge
8	import androidx.activity.viewModels
9	import androidx.compose.foundation.Image
10	import androidx.compose.foundation.layout.*
11	import androidx.compose.foundation.lazy.LazyColumn
12	import androidx.compose.foundation.lazy.items
13	import androidx.compose.foundation.shape.RoundedCornerShape
14	import androidx.compose.material3.*
15	import androidx.compose.runtime.Composable
16	import androidx.compose.runtime.collectAsState
17	import androidx.compose.runtime.getValue
18	import androidx.compose.ui.Alignment
19	import androidx.compose.ui.Modifier
20	import androidx.compose.ui.draw.clip
21	import androidx.compose.ui.graphics.Color
22	import androidx.compose.ui.layout.ContentScale
23	import androidx.compose.ui.platform.LocalContext
24	import androidx.compose.ui.res.painterResource
25	import androidx.compose.ui.res.stringResource
26	import androidx.compose.ui.text.TextStyle
27	import androidx.compose.ui.text.font.FontWeight
28	import androidx.compose.ui.unit.dp
29	import androidx.compose.ui.unit.sp
30	import androidx.core.net.toUri
31	import com.example.mlistcompose.model.ListMovie
32	import com.example.mlistcompose.ui.theme.MListComposeTheme
33	import timber.log.Timber
34	
35	class MainActivity : ComponentActivity() {
36	
37	private val viewModel: MovieViewModel by viewModels {
38	MovieViewModelFactory("Test Debugging")
39	}
40	
41	override fun onCreate(savedInstanceState: Bundle?) {
42	super.onCreate(savedInstanceState)
43	enableEdgeToEdge()
44	Timber.plant(Timber.DebugTree())

```

45         setContent {
46             MListComposeTheme {
47                 Surface(
48                     modifier = Modifier.fillMaxSize(),
49                     color = Color.Black
50                 ) {
51                     MovieListScreen(viewModel)
52                 }
53             }
54         }
55     }
56 }
57
58 @Composable
59 fun MovieListScreen(viewModel: MovieViewModel) {
60     val movieList by viewModel.movies.collectAsState()
61     val navigateToUrl by
62     viewModel.navigateToUrl.collectAsState()
63     val navigateToDetail by
64     viewModel.navigateToDetail.collectAsState()
65     val context = LocalContext.current
66
67     navigateToUrl?.let { url ->
68         val intent = Intent(Intent.ACTION_VIEW, url.toUri())
69         context.startActivity(intent)
70         viewModel.onUrlNavigated()
71     }
72
73     navigateToDetail?.let { movie ->
74         val intent = Intent(context,
75         DetailActivity::class.java).apply {
76             putExtra("title", movie.title)
77             putExtra("years", movie.years)
78             putExtra("image", movie.image)
79             putExtra("description", movie.description)
80         }
81         context.startActivity(intent)
82         viewModel.onDetailNavigated()
83     }
84
85     LazyColumn(
86         contentPadding = PaddingValues(16.dp),
87         verticalArrangement = Arrangement.spacedBy(16.dp)
88     ) {
89         items(movieList) { movie ->
90             MovieItems(
91                 movie = movie,
92                 onImdbClicked = {
93                     viewModel.onLinkClicked(movie.url) },
94                 onDetailClicked = {
95                     viewModel.onDetailClicked(movie) }
96             )
97         }
98     }
99 }

```



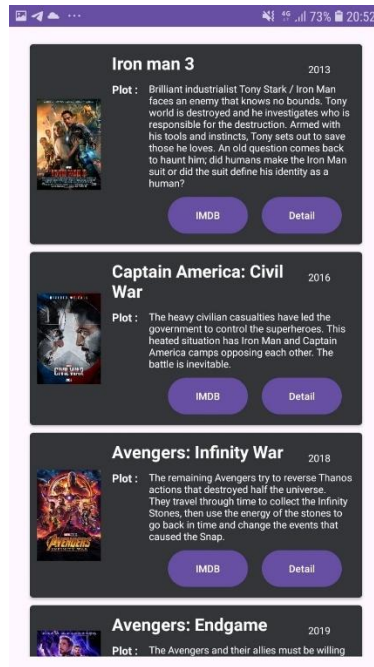
```

143         color = Color.White
144     ),
145     modifier = Modifier
146         .padding(bottom = 4.dp)
147         .weight(1f)
148     )
149     Text(
150         text = releaseDate,
151         style = TextStyle(fontSize = 20.sp,
color = Color.White),
152         modifier = Modifier
153             .padding(bottom = 4.dp)
154             .wrapContentWidth(Alignment.End)

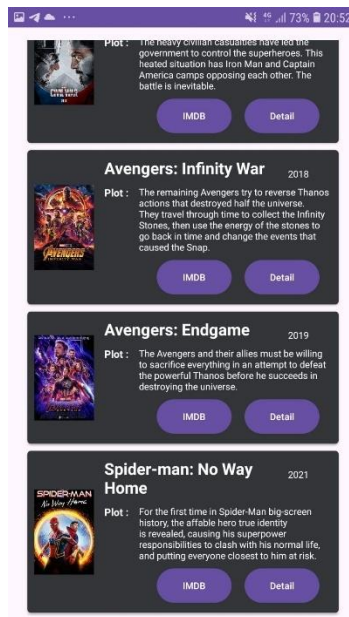
155     .align(Alignment.CenterVertically)
156     )
157     }
158     Row(
159         modifier = Modifier.fillMaxWidth()
160     ) {
161         Text(
162             text = "Plot :",
163             style = TextStyle(
164                 fontSize = 16.sp,
165                 fontWeight = FontWeight.Bold,
166                 color = Color.White
167             )
168         )
169         Text(
170             text = stringResource(movie.desc),
171             style = TextStyle(fontSize = 16.sp,
color = Color.White),
172             modifier = Modifier.padding(bottom =
8.dp)
173         )
174     }
175     Row(
176         modifier = Modifier.fillMaxWidth(),
177         horizontalArrangement = Arrangement.End
178     ) {
179         Button(onClick = onImdbClicked) {
180             Text("IMDB")
181         }
182         Spacer(modifier = Modifier.width(4.dp))
183         Button(onClick = onDetailClicked) {
184             Text("Detail")
185         }
186     }
187 }
188 }
189 }
190 }

```

B. Output Program



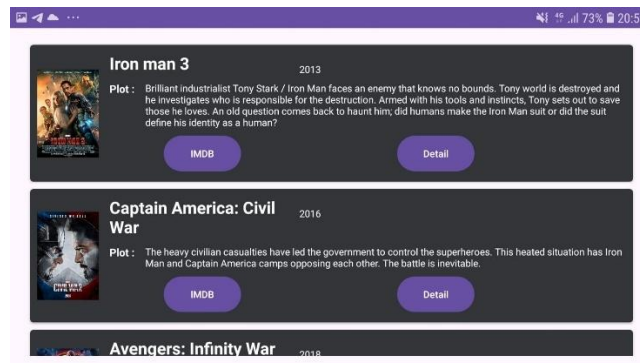
Gambar 1. Tampilan Awal UI XML



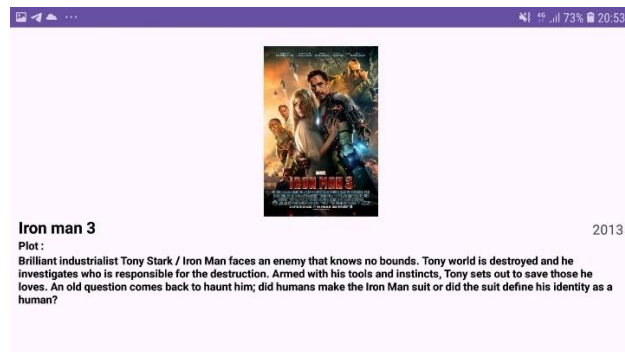
Gambar 2. Tampilan Ketika di Scroll XML



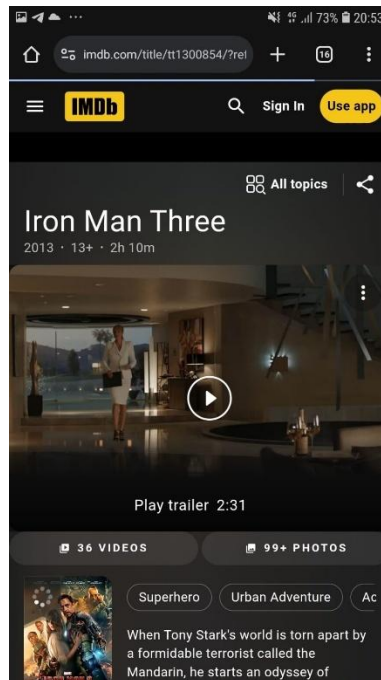
Gambar 3. Tampilan Tombol Detail Ditekan XML



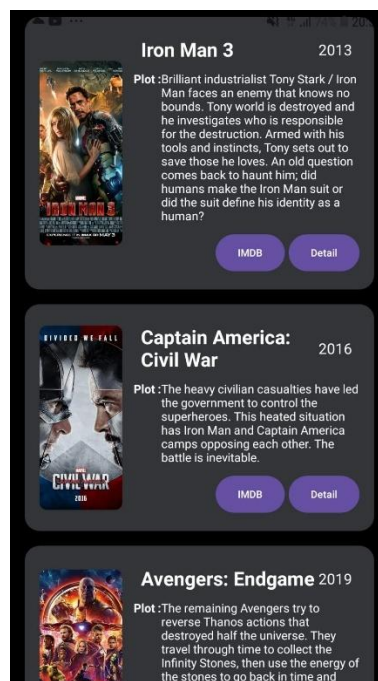
Gambar 4. Tampilan Awal UI di Rotate XML



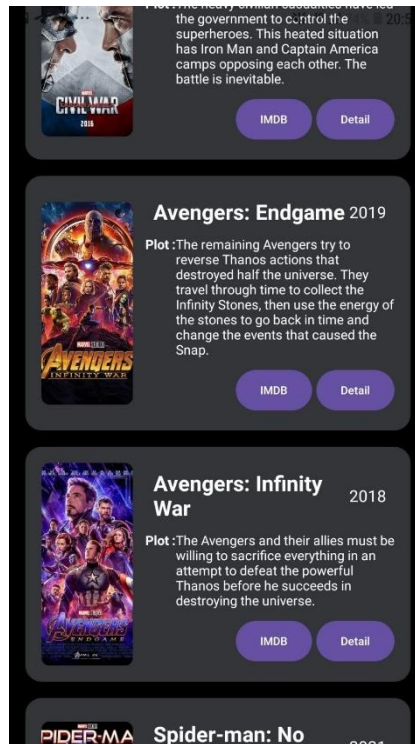
Gambar 5. Tampilan Detail UI di Rotate XML



Gambar 6. Tombol IMDB di Tekan



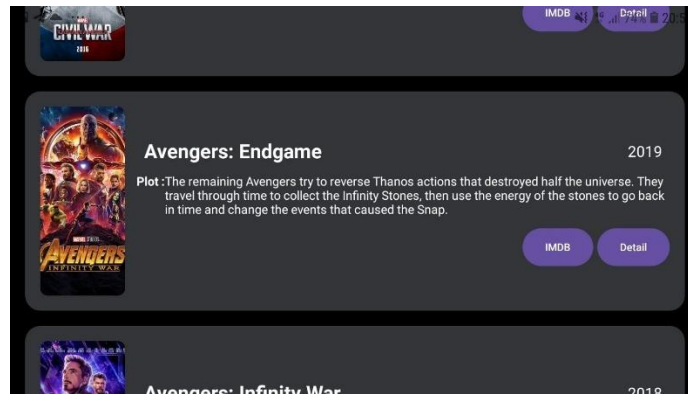
Gambar 7. Tampilan Awal UI Jetpack Compose



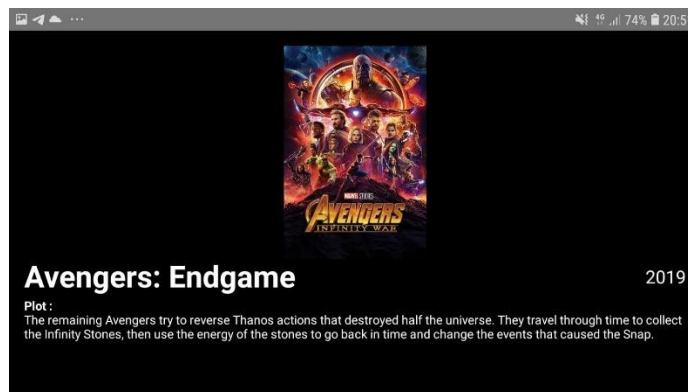
Gambar 8. Tampilan UI Ketika di Scroll Jetpack Compose



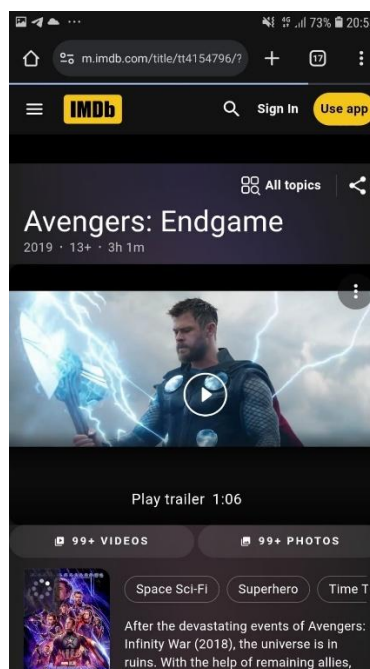
Gambar 9. Tampilan Detail UI Jetpack Compose



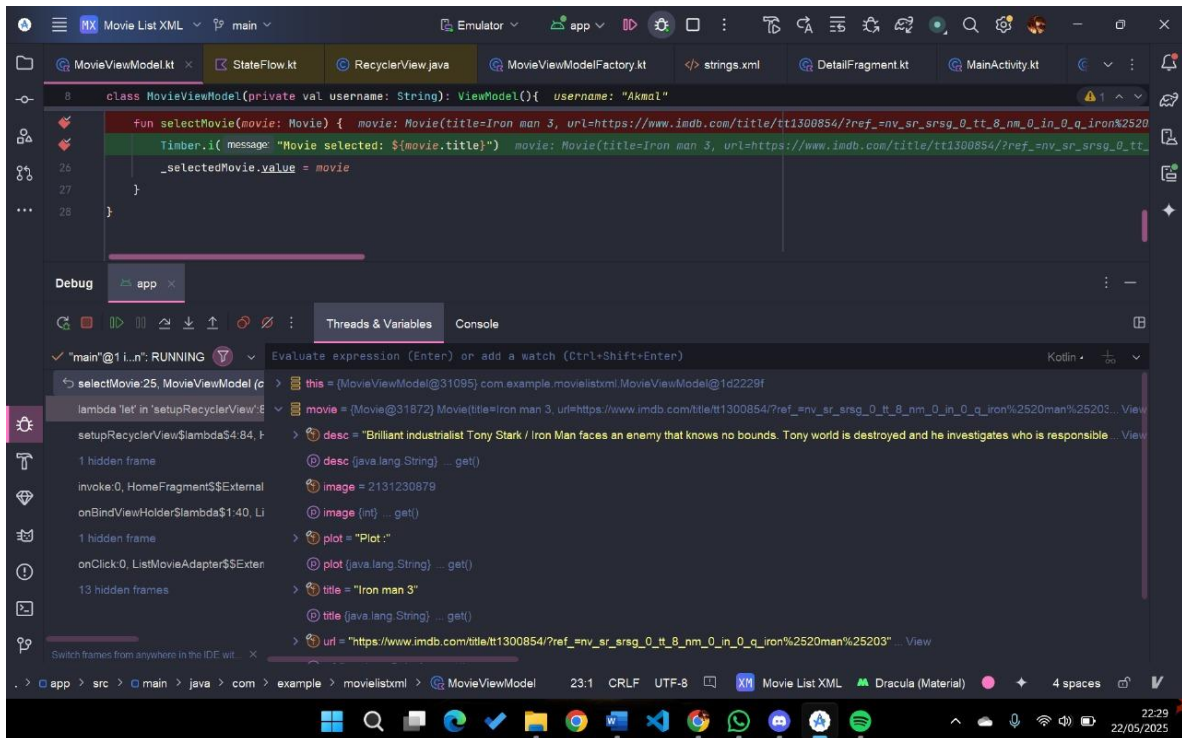
Gambar 10. Tampilan Awal UI di Rotate Jetpack Compose



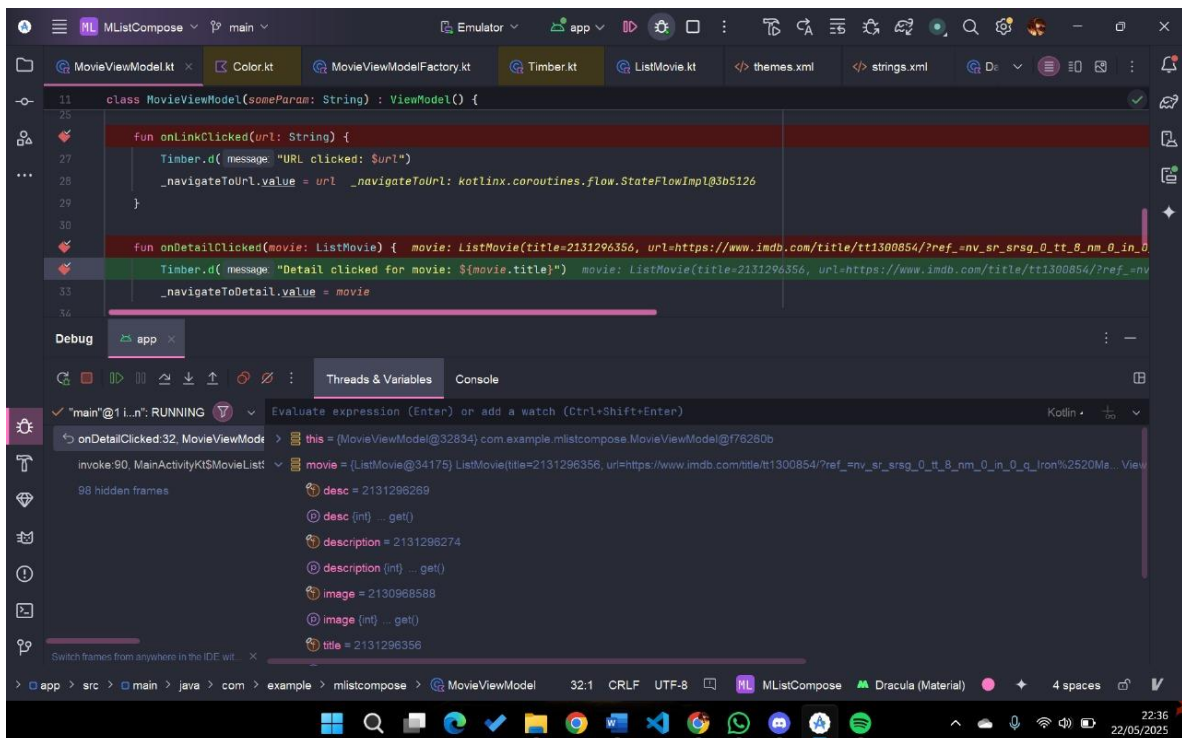
Gambar 11. Tampilan Detail UI di Rotate Jetpack Compose



Gambar 12. Tampilan Ketika Tombol IMDB di Tekan Jetpack Compose



Gambar 13. Gambar Debugger Di XML



Gambar 14. Gambar Debugger Di Jetpack Compose

C. Pembahasan

XML

MainActivity.kt

Pada baris [1 – 5], mendeklarasikan package dan import yang digunakan untuk definisi file merupakan bagian dari package com.example.movielistxml

Pada baris [7], mendeklarasikan activity utama dari AppCompatActivity, yang merupakan MainActivity adalah layar utama aplikasi

Pada baris [8 – 23], method onCreate mengatur layout activity dengan setContentView, lalu menggunakan fragmentManager untuk memeriksa apakah HomeFragment sudah ada. Jika belum ada, HomeFragment akan dibuat dan ditambahkan ke container frame_container dengan tag nama kelasnya, serta mencatat log debug menggunakan Timber.

Movie.kt

Pada baris [1 – 4]. Mendeklarasikan file dalam package yang sama dan interface untuk memungkinkan objek dikirim antar komponen android antar activity atau fragment. Parcelize untuk anotasi dari Kotlin extension untuk menghindari penulisan parcelable secara manual.

Pada baris [6 – 14], menginstruksikan Kotlin secara otomatis menghasilkan parcelable untuk kelas ini dan data yang digunakan untuk menyimpan dan mengelola data secara sederhana. Terdapat beberapa properti seperti title, url, image, plot, years, dan desc. Parcelable menandakan bahwa objek movie bisa diparcel dan dikirim dengan intent atau bundle.

ListMovieAdapter.kt

Pada baris [1 – 9], package aplikasi dideklarasikan dan beberapa kelas Android seperti LayoutInflater, View, ViewGroup, Button, ImageView, TextView, serta RecyclerView diimpor untuk membuat adapter daftar film menggunakan RecyclerView. Pada baris [12 – 16], berisi class yang berisi listMovie sebagai daftar film yang akan ditampilkan, lambda function yang menerima url dan detail film saat di tekan. Selain itu mewarisi RecyclerView.Adapter dan ViewHolder sebagai ViewHoldernya

Pada baris [11 – 15], dibuat kelas ListMovieAdapter yang merupakan adapter RecyclerView untuk menampilkan daftar film dengan parameter listMovie sebagai data film dan dua fungsi callback untuk tombol IMDB dan Detail agar bisa mengirim data saat tombol diklik.

Pada baris [17 - 25], dibuat kelas ViewHolder yang menyimpan referensi ke elemen-elemen tampilan di layout item RecyclerView, seperti gambar poster, judul film, tahun, plot, deskripsi, dan dua tombol untuk IMDB serta Detail.

Pada baris [26 - 29], fungsi onCreateViewHolder membuat dan mengembalikan objek ViewHolder dengan meng-inflate layout item_movie sebagai tampilan tiap item dalam RecyclerView.

Pada baris [30], berfungsi untuk mengembalikan jumlah item dalam listMovie, yang digunakan RecyclerView untuk mengetahui berapa banyak item yang harus ditampilkan.

Pada baris [32 - 41], fungsi `onBindViewHolder` mengatur data film ke tampilan item `RecyclerView`, seperti judul, plot, gambar, tahun, dan deskripsi. Tombol IMDb dan Detail diberikan aksi klik untuk memicu fungsi yang mengirim data URL dan detail film.

Pada baris [43 - 47], fungsi `updateData` digunakan untuk memperbarui daftar film di adapter dengan data baru, lalu memberi tahu `RecyclerView` agar tampilan diperbarui sesuai data terbaru.

HomeFragment.kt

Pada baris [1 - 16], terdapat import berbagai library yang dibutuhkan untuk fragment, pengelolaan lifecycle, tampilan `RecyclerView`, data binding, coroutine, dan logging menggunakan Timber.

Pada baris [18 - 27], didefinisikan kelas `HomeFragment` yang merupakan subclass `Fragment`. Ada variabel `_binding` untuk binding layout yang diakses lewat properti binding. Juga dideklarasikan `movieAdapter` untuk `RecyclerView` dan `viewModel` yang diinisialisasi menggunakan `MovieViewModelFactory` dengan parameter "Akmal".

Pada baris [29 - 35], fungsi `onCreateView` dibuat untuk meng-inflate layout fragment menggunakan `FragmentHomeBinding`. Binding ini disimpan di `_binding`, lalu mengembalikan `binding.root` sebagai tampilan fragment yang akan ditampilkan.

Pada baris [37 - 70], fungsi `onViewCreated` mengatur `RecyclerView` dengan memanggil `setupRecyclerView()`. Kemudian data film diambil dari `getListMovie()` dan dikirim ke `ViewModel`. Dua coroutine diluncurkan untuk mengamati perubahan data film (`movieList`) dan film yang dipilih (`selectedMovie`). Saat daftar film diperbarui, adapter diperbarui dengan data baru. Jika ada film yang dipilih, aplikasi akan membuat dan menampilkan `DetailFragment` dengan data film tersebut, lalu mengganti fragment saat ini dengan fragment detail dan menambahkan transaksi ke back stack.

Pada baris [73 - 94], fungsi `setupRecyclerView` membuat adapter `ListMovieAdapter` dengan list kosong dan dua callback: saat tombol IMDb diklik, aplikasi membuka browser ke URL film, dan saat tombol Detail diklik, aplikasi mencari film berdasarkan judul lalu memilih film tersebut di `ViewModel`. Setelah itu, `RecyclerView` diatur dengan `LinearLayoutManager`, adapter yang sudah dibuat, dan pengaturan ukuran tetap agar performa lebih baik.

Pada baris [96 - 110], fungsi `getListMovie()` mengambil data film dari resource seperti judul, link, gambar, plot, tahun, dan deskripsi. Data tersebut dimasukkan ke dalam list `ArrayList<Movie>` secara berurutan menggunakan perulangan, lalu list ini dikembalikan sebagai hasil fungsi. `dataImage.recycle()` dipanggil untuk membebaskan resource gambar setelah digunakan.

Pada baris [112 - 115], `onDestroyView()` dipanggil saat tampilan fragment dihancurkan, dan di sini variabel binding di-set ke null untuk menghindari memory leak dengan membersihkan referensi ke view.

DetailFragment.kt

Pada baris [1 - 8], ini adalah deklarasi paket dan impor kelas penting yang dibutuhkan untuk membuat Fragment di Android, termasuk `FragmentDetailBinding` yang menghubungkan layout XML fragment detail dengan kode Kotlin agar bisa mengakses elemen UI secara langsung.

Pada baris [10 - 13], dibuat kelas `DetailFragment` yang merupakan subclass dari `Fragment`. Di sini juga dideklarasikan variabel `_binding` yang bertipe nullable `FragmentDetailBinding` untuk mengelola binding layout, serta properti binding yang non-nullable agar memudahkan akses elemen UI dengan aman setelah binding diinisialisasi.

Pada baris [15 - 36], method `onCreateView` di-override untuk membuat dan menghubungkan tampilan fragment. Binding layout diinisialisasi dengan `FragmentDetailBinding.inflate`. Data film seperti judul, gambar, tahun, plot, dan deskripsi diambil dari argumen fragment dengan key yang sesuai. Nilai tersebut kemudian ditampilkan di elemen UI menggunakan binding, misalnya `tvItemTitle`, `tvItemPlot`, `tvItemYears`, `tvItemDesc`. Gambar dipasang jika ada nilai image. Akhirnya, root view dari binding dikembalikan sebagai tampilan fragment. Ada kesalahan kecil pada penempatan data: `tvItemPlot` diisi dengan years, dan `tvItemYears` diisi dengan plot (seharusnya dibalik).

Pada baris [37 - 40], method `onDestroyView` di-override untuk membersihkan binding ketika view fragment dihancurkan. Ini penting untuk menghindari memory leak dengan menyetel `_binding` menjadi null saat view sudah tidak digunakan lagi.

MovieViewModel.kt

Pada baris [1 - 6], kode mengimpor paket utama untuk membuat `ViewModel`, mengelola `StateFlow` dari Kotlin Coroutines untuk data yang dapat diobservasi, dan `Timber` untuk logging/debugging di aplikasi Android.

Pada baris [8 - 13], kelas `MovieViewModel` mendefinisikan dua properti `MutableStateFlow` privat untuk menyimpan daftar film (`_movieList`) dan film yang dipilih (`_selectedMovie`). Properti publik `movieList` dan `selectedMovie` mengexpose `StateFlow` hanya baca untuk pengamatan dari luar kelas. Konstruktor menerima parameter username yang disimpan sebagai properti kelas.

Pada baris [15 - 17], blok init akan langsung dijalankan saat instance `MovieViewModel` dibuat, di mana terdapat logging menggunakan `Timber` untuk mencatat pembuatan `ViewModel` beserta username yang diterima.

Pada baris [19 - 22], fungsi `setMovieList` menerima parameter `movies` berupa daftar `Movie`, mencatat jumlah item yang diterima menggunakan `Timber`, lalu memperbarui nilai `_movieList` dengan daftar tersebut.

Pada baris [24 - 27], fungsi `selectMovie` menerima objek `movie` bertipe `Movie`, mencatat judul film yang dipilih dengan `Timber`, lalu menyimpan objek `movie` tersebut ke dalam `_selectedMovie` untuk menandai film yang sedang dipilih.

MovieViewModelFactory.kt

Pada baris [1 - 3], package `com.example.movieslistxml` dideklarasikan, lalu `ViewModel` dari `androidx.lifecycle` diimpor untuk digunakan sebagai kelas dasar dalam membuat kelas `ViewModel` yang bertugas mengelola data dan logika UI secara lifecycle-aware.

Pada baris [5 – 13], didefinisikan kelas `MovieViewModelFactory` yang mengimplementasikan `ViewModelProvider.Factory`. Fungsi `create` akan membuat instance `MovieViewModel` dengan parameter `username` jika `modelClass` sesuai. Jika tidak, akan melempar `IllegalArgumentException` karena kelas `ViewModel` tidak dikenal. Ini digunakan untuk membuat `ViewModel` dengan konstruktor yang membutuhkan argumen.

MyApplication.kt

Pada baris [1 - 13], kode ini mendefinisikan kelas `MyApplication` yang merupakan subclass dari `Application`. Di dalam metode `onCreate()`, kode memanggil `super.onCreate()` untuk menjalankan inisialisasi standar aplikasi. Kemudian, menggunakan pengecekan `BuildConfig.DEBUG` untuk memastikan bahwa fitur logging dengan `Timber` hanya diaktifkan saat aplikasi berjalan dalam mode debug. Jika benar, `Timber.DebugTree()` dipasang agar log dari aplikasi bisa dicatat dan ditampilkan di `Logcat` selama pengembangan. Ini membantu developer memonitor jalannya aplikasi dengan mudah tanpa menampilkan log di versi rilis.

***activity_main.xml**

Pada baris [1 – 9], membuat ID di Kotlin `MainActivity` sebagai tempat menampung fragment, `match parent` akan mengisi seluruh layar dan `contextnya` akan terkait dengan `MainActivity`

Terdapat 2 `activity_activity main` dengan isi yang sama tapi salah satu nya itu berfungsi sebagai `landscape` saat layar di rotate

***fragment_detail.xml**

Pada baris [10 – 19], `<ImageView` layout ini akan digunakan untuk penempatan gambar khusus poster film dengan id `img_poster` yang digunakan pada kode Kotlin. Peletakkannya di bagian atas layout dan rata kiri

Pada baris [21 – 31], `<TextView` layout ini digunakan untuk penempatan title dengan id `tv_item_title` yang berada tepat di bawah poster.

Pada baris [33 – 43], `<TextView` layout ini digunakan untuk penempatan tahun dengan id `tv_item_years` yang terletak di sebelah kanan judul

Pada baris {45 – 55}, `<TextView` layout ini digunakan untuk penempatan label teks sebelum deskripsi plot dengan id `tv_item_plot`

Pada baris [57 – 67], `<TextView` layout ini digunakan untuk penempatan deskripsi film dengan id `tv_item_desc`

Terdapat 2 fragment_detail juga dengan isi yang sama tapi yang satu berfungsi untuk menjaga tampilan saat landscape atau layar di rotate

***fragment_home.xml**

Pada baris [1 – 19], membuat container fleksibel untuk penempatan berdasarkan constraint view dan ke parent, id rv_movie diakses di HomeFragment untuk menampilkan list movie dengan beberapa aturan lebar, tinggi, margin dan layout constraint untuk RecyclerView memenuhi semua area parent dengan margin

***item_movie.xml**

Pada baris [1 – 19], membuat constraint layout CardView untuk mengatur posisi elemen secara fleksible dan memberi warna latar abu abu

Pada baris [21 – 29], <ImageView untuk membuat layout gambar dengan scaletype center agar gambar memenuhi area dan constraint ke atas, bawah, kiri parent

Pada baris [31 – 43], <TextView untuk membuat judul film meletakkannya disebelah kanan poster dan diapit img poster dan tv_item_years

Pada baris [45 – 55], <TextView untuk membuat tahun film yang peletakkannya di kanan judul dan menampilkan rilis filmnya kapan.

Pada baris [57 – 68], <TextView untuk membuat label plot di bawah judul dan dikanan poster

Pada baris [70 – 80], <TextView untuk membuat deskripsi film di sebelah label plot dan menampilkan ringkasan cerita film

Pada baris [82 – 91], <Button untuk tombol detail pindah ke DetailFragment di bawah deskripsi, dikanan bersebelahan dengan tombol IMDB

Pada baris [93 – 101], <Button untuk tombol pindah halaman ke link IMDB dan terletak di bawah deskripsi di kiri.

Jetpack Compose

***strings.xml**

Pada string ini, berisikan data data yang akan digunakan pada kode Kotlin yang akan dijelaskan setelahnya. StringArray ini berfungsi sebagai data yang akan digunakan untuk menampilkan film atau deskripsi dan semacamnya.

DataMovie.kt

Pada baris [1 – 4], menempatkan file di package data, mengimpor R untuk akses ke resource seperti R.string atau R.drawable. mengimpor ListMovie dari class model yang merepresentasikan satu film

Pada baris [6 -13], fungsi `loadMovieList()` bertugas menghasilkan daftar data film dalam bentuk `List<ListMovie>`. Fungsi ini mengembalikan data statis yang berisi beberapa objek `ListMovie`, di mana setiap objek memuat informasi seperti judul film (title), tautan IMDb (url), poster (image), tanggal rilis (releaseDate), ringkasan (desc), dan deskripsi lengkap (description). Nilai-nilai ini tidak ditulis langsung dalam bentuk string atau gambar, melainkan mengacu pada ID dari resource yang disimpan di `strings.xml` dan `drawable`. Pendekatan ini memudahkan pengelolaan konten dan mendukung penerjemahan jika dibutuhkan. Data hasil fungsi ini nantinya akan digunakan di UI, seperti pada `LazyColumn`, untuk menampilkan daftar film secara dinamis berdasarkan resource yang telah ditentukan.

ListMovie.kt

Pada baris [1 – 4], menempatkan file di package model karena representasi data, `@StringRes` dan `@DrawableRes` untuk anotasi validasi tipe resource

Pada baris [6 – 13], membuat data class untuk menyimpan data film yang masing masing bertipe data `Int` karena akan mengambil dari resource ID. Properti image dianotasi dengan `@DrawableRes` karena nilainya merujuk pada ID gambar di `drawable`. Sementara itu, properti url bertipe `String` tanpa anotasi karena nilainya berupa URL langsung dan bukan ID resource. Ini mencerminkan bahwa URL tidak diambil dari `strings.xml`, melainkan ditulis langsung dalam bentuk string literal.

DetailActivity.kt

Pada baris [1 – 19], membuat activity dasar compose dan mengimpor lainnya seperti elemen UI compose seperti `Image`, `Text`, `Column`, `Row` dan sebagainya

Pada baris [21 – 22], membuat activity yang digunakan untuk menampilkan detail film yang dipilih dari list, dan `onCreate` untuk format awal eksekusi UI

Pada baris [24 – 27], mengambil data dari intent dan data dikirim dalam bentuk resource ID sesuai `@StringRes` dan `@DrawableRes`

Pada baris [29 – 34], surface untuk memberi tempat dengan latar hitam dan `MListComposeTheme` membungkus UI agar mengikuti tema aplikasi

Pada baris [35 – 40], menggunakan `column` untuk menampilkan elemen secara vertikal

Pada baris [41 – 45], menampilkan gambar dari resource ID dan `contentDescription` diisi dengan judul film untuk aksesibilitas

Pada baris [46 – 70], menggunakan `row` agar judul dan tahun sejajar secara horizontal dan `weight 1F` pada judul agar mengisi ruang, dan tahun mengikuti di kanan

Pada baris [71 – 85], `Text` yang akan digunakan untuk label dan isi deskripsi film

MainActivity.kt

Pada baris [1 – 33], merupakan bagian awal dari file yang menandakan bahwa file ini berada dalam package `com.example.mlistcompose`. Bagian ini juga mengimpor berbagai komponen

dari Jetpack Compose, Android framework, tema MListComposeTheme, dan pustaka eksternal seperti Timber untuk keperluan logging. Pada baris [48 – 63], activity utama yang memanggil MovieListScreen() saat app dibuka dengan menggunakan tema compose dan meletakkan di surface untuk interface dengan background hitam. Data list film diambil dari DataMovie().loadMovieList()

Pada baris [35 – 56], kelas MainActivity berfungsi sebagai activity utama aplikasi yang menggunakan Jetpack Compose. ViewModel MovieViewModel diinisialisasi untuk mengatur data yang akan ditampilkan. Di dalam fungsi onCreate, enableEdgeToEdge() dipanggil agar tampilan memenuhi layar, lalu Timber.plant() digunakan untuk debugging. Selanjutnya, setContent menampilkan UI dengan tema MListComposeTheme, kemudian seluruh isi layar dibungkus Surface berwarna hitam dan menampilkan MovieListScreen sebagai tampilan utama yang menampilkan daftar film.

Pada baris [58 – 64], fungsi MovieListScreen adalah composable yang menampilkan daftar film menggunakan data dari MovieViewModel. Fungsi ini mengambil state film, navigasi ke URL, dan navigasi ke detail dengan collectAsState(). Jika ada URL untuk dibuka, akan memicu intent browser, sedangkan jika ada film untuk detail, akan membuka DetailActivity dengan data film dikirim lewat intent. Daftar film ditampilkan dalam LazyColumn dengan jarak antar item, di mana setiap item dipanggil melalui MovieItems yang menerima aksi klik untuk membuka link IMDB atau detail film.

Pada baris [96 – 189], fungsi MovieItems adalah composable yang menampilkan satu item film dalam bentuk kartu dengan desain sudut membulat dan warna gelap. Di dalam kartu ini, ada gambar film yang ditampilkan dengan ukuran khusus dan diapit oleh teks judul dan tahun rilis yang diambil dari resource string. Di bawah judul, ada deskripsi singkat plot film. Di bagian bawah kartu terdapat dua tombol, yaitu tombol "IMDB" untuk membuka link film di browser dan tombol "Detail" untuk membuka halaman detail film. Tata letak menggunakan kombinasi Row dan Column agar elemen tampil rapi dan proporsional dengan padding dan jarak antar elemen.

MovieViewModel.kt

Pada baris [1 – 9], ini adalah ViewModel yang mengelola data film dan navigasi menggunakan StateFlow, serta memakai Timber untuk logging debug.

Pada baris [11 – 19], kelas MovieViewModel menyimpan daftar film dan status navigasi menggunakan MutableStateFlow yang bersifat private, serta menyediakan akses publik lewat StateFlow agar data bisa dipantau tanpa bisa diubah langsung dari luar.

Pada baris [20 – 24], blok init menjalankan kode saat ViewModel dibuat dengan mencatat log proses loading data film menggunakan Timber, lalu memuat daftar film dari DataMovie ke dalam _movies, dan mencatat jumlah item yang berhasil dimuat.

Pada baris [26 – 29], fungsi onLinkClicked mencatat log saat URL diklik dan mengatur nilai _navigateToUrl dengan URL tersebut untuk memicu navigasi ke browser.

Pada baris [31 – 34], fungsi `onDetailClicked` mencatat log saat detail film diklik dan mengatur nilai `_navigateToDetail` dengan data film tersebut untuk memicu navigasi ke halaman detail.

Pada baris [36 – 42], fungsi `onUrlNavigated` dan `onDetailNavigated` digunakan untuk mengatur ulang nilai navigasi menjadi null setelah navigasi selesai, agar event navigasi tidak terjadi berulang.

MovieViewModelFactory.kt

Pada baris [1 – 4], package didefinisikan sebagai `com.example.mlistcompose` dan mengimpor kelas `ViewModel` serta `ViewModelProvider` dari `AndroidX Lifecycle` untuk mendukung pembuatan `ViewModel` dengan parameter khusus.

Pada baris [6 – 14], dibuat kelas `MovieViewModelFactory` yang bertugas membuat instance `MovieViewModel` dengan parameter `someParam`, menggunakan override fungsi `create` dari `ViewModelProvider.Factory` untuk mengontrol pembuatan `ViewModel` dan memastikan tipe yang benar, serta melemparkan exception jika kelas `ViewModel` tidak dikenali.

Step Into, Step Over, dan Step Out:

Debugger adalah alat yang penting untuk pengembangan aplikasi, dengan debugger kita bisa memahami alur eksekusi kode, memeriksa nilai variable, mengidentifikasi bug, dan menguji kondisi tertentu. Pada gambar terdapat beberapa breakpoint yang menghentikan eksekusi program, pada gambar juga terlihat breakpoint akan menghasilkan apa yang terjadi atau muncul saat item di klik atau diakses. Di window debug kita akan melihat beberapa seperti frame yang menunjukkan tumpukan fungsi, variables, threads dan console.

Fitur Step Over pada gambar jika berada pada baris 27 akan meloncat ke baris 28 atau 29 tanpa metode internal `Timber.d`

Step Into, pada gambar memungkinkan membawa ke kode sumber dari fungsi `timber.d`, jika berada di baris 29 atau `navigate to detail url` akan membawa ke baris pertama dari metode

Step Out, jika kita menggunakan Step Into dan ada di implementasi `timber.d` dan step out ditekan, akan mengembalikan ke baris panggilan `Timber.d` di `ModelViewModel.kt`

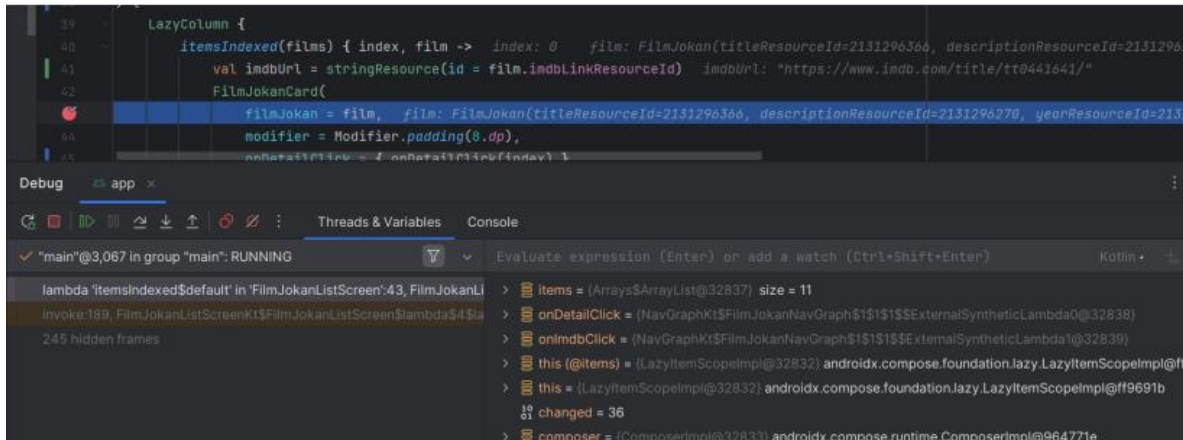
D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Akmlsybn/Praktikum-Mobile>

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



A. Pembahasan

Dalam android, application class adalah kelas dasar untuk menjaga state global aplikasi. Ini adalah singleton di seluruh siklus hidup atau lifecycle aplikasi. Application class dibuat ketika proses aplikasi dimulai dan akan dihancurkan ketika proses dimatikan. Fungsinya adalah menginisialisasi global, menyimpan data yang tersedia di seluruh komponen, dan class ini memiliki callback yang dipanggil saat aplikasi pertama kali dibuat.

Secara singkat dalam gambar ini menjaga fitur seperti verifikasi data yang dibuat memiliki nilai, jika ada kesalahan maka akan bisa dilacak ke sumber data.